

CYBER INCIDENT HUB

Technical Backlog — Tickets por Capas

Generado para: Proyecto Final Full Stack · CI4 REST API + React

15 tickets · 4 capas · estimado total: 7 a 10 días de trabajo con IA

Resumen de tickets y tiempos

ID	Ticket	Capa	Tiempo estimado
DB-01	Schema MySQL completo	CAPA 1	3-4 hs
API-01	Setup CI4 como API REST	CAPA 2	2-3 hs
API-02	Entity / DTO / Converter	CAPA 2	3-4 hs
API-03	JWT Service completo	CAPA 2	3-4 hs
API-04	Auth: register, login, me	CAPA 2	3-4 hs
API-05	Incidents CRUD + filtros	CAPA 2	5-6 hs
API-06	Likes, favoritos, comentarios	CAPA 2	3-4 hs
API-07	Dashboard endpoints	CAPA 2	2-3 hs
API-08	Admin panel endpoints	CAPA 2	2-3 hs
FE-01	Setup React + Auth Context	CAPA 3	3-4 hs
FE-02	Login y Register pages	CAPA 3	3-4 hs
FE-03	Feed + filtros + paginación	CAPA 3	4-5 hs
FE-04	Detalle + Dashboard + Admin	CAPA 3	5-6 hs
INT-01	Testing end-to-end	CAPA 4	3-4 hs
INT-02	Pulido y detalles finales	CAPA 4	3-4 hs

🕒 Con 6 horas diarias y la IA como aliada, el estimado total es de 7 a 10 días de trabajo real. Los primeros 3 días son los más densos (DB + JWT + estructura base). A partir del día 4 empieza a fluir porque el patrón se repite.

CAPA 1 — Base de datos

Diseñar el schema de la base de datos. Es el cimiento de todo. Un campo mal pensado aquí cuesta mucho más arreglarlo después.

DB-01 · Diseño y creación del schema MySQL completo

Base de datos

CRÍTICO

3-4
hs

OBJETIVO

Crear el script SQL con todas las tablas del sistema: users, incidents, categories, comments, tags, incident_tags (M:M), likes, favorites. Este script es la base de todo el proyecto.

CONTEXTO

El diseño de la DB define todas las relaciones del sistema. Una vez que haya código encima, cambiar la estructura es costoso. Dedícale tiempo a pensar bien los campos antes de avanzar. Toda la información "única por usuario" (likes, favoritos, mis posts) depende de que user_id esté correctamente como FK en cada tabla.

JWT INVOLUCRADO

No aplica directamente aquí, pero el campo "role" de la tabla users es el que el JWT va a cargar en sus claims. Definí bien los valores posibles: "user" y "admin".

ARCHIVOS A CREAR / MODIFICAR

- > database/migrations/001_create_users.sql
- > database/migrations/002_create_categories.sql
- > database/migrations/003_create_incidents.sql
- > database/migrations/004_create_comments.sql
- > database/migrations/005_create_tags.sql
- > database/migrations/006_create_incident_tags.sql
- > database/migrations/007_create_likes.sql
- > database/migrations/008_create_favorites.sql
- > database/seeds/CategorySeeder.php

PASOS DETALLADOS

- > Tabla users: id (PK, AUTO_INCREMENT), username (VARCHAR 50, UNIQUE), email (VARCHAR 100, UNIQUE), password_hash (VARCHAR 255), role (ENUM "user","admin", DEFAULT "user"), avatar_url (VARCHAR 255, NULLABLE), bio (TEXT, NULLABLE), is_banned (TINYINT 1, DEFAULT 0), created_at, updated_at.
- > Tabla categories: id, name (VARCHAR 50, UNIQUE), slug (VARCHAR 50, UNIQUE), description (TEXT), icon (VARCHAR 50). Ejemplos de categorías: Ransomware, Zero-Day, Phishing, DDoS, Data Breach, Malware.
- > Tabla incidents: id, user_id (FK → users.id), category_id (FK → categories.id), title (VARCHAR 255), slug (VARCHAR 255, UNIQUE), description (TEXT), severity (ENUM: "critical","high","medium","low"), status (ENUM: "active","resolved","archived", DEFAULT "active"), ip_address (VARCHAR 45, NULLABLE), domain (VARCHAR 255, NULLABLE), cve_id (VARCHAR 20, NULLABLE — formato CVE-YYYY-NNNNN), malware_name (VARCHAR 100, NULLABLE), country (VARCHAR 2, NULLABLE — ISO 3166), is_featured (TINYINT 1, DEFAULT 0), views_count (INT, DEFAULT 0), created_at, updated_at.
- > Tabla comments: id, incident_id (FK → incidents.id), user_id (FK → users.id), content (TEXT), is_deleted (TINYINT 1, DEFAULT 0 — soft delete), created_at, updated_at.
- > Tabla tags: id, name (VARCHAR 50, UNIQUE), slug (VARCHAR 50, UNIQUE).
- > Tabla incident_tags: incident_id (FK), tag_id (FK). PK compuesta: (incident_id, tag_id). Es la tabla pivote de la relación M:M entre incidents y tags.
- > Tabla likes: user_id (FK), incident_id (FK). PK compuesta: (user_id, incident_id). Un usuario solo puede likear un incidente una vez.
- > Tabla favorites: user_id (FK), incident_id (FK). PK compuesta: (user_id, incident_id). Igual que likes pero semánticamente distinto (guardar para leer después).
- > Agregar índices donde corresponda: incidents.slug, incidents.severity, incidents.category_id, incidents.user_id, incidents.cve_id, comments.incident_id.

- › Crear el seeder de categorías con al menos 6 categorías iniciales para poder testear filtros desde el primer día.

CRITERIO DE ACEPTACIÓN — "esto funciona cuando..."

- › Correr el script completo en MySQL sin errores.
- › Verificar con SHOW CREATE TABLE que las FK y constraints quedaron bien.
- › Insertar un usuario de prueba y un incidente manualmente para verificar las relaciones.
- › El seeder de categorías corre sin errores y se ven los registros en la tabla.

NOTA

Usá VARCHAR(45) para ip_address para soportar IPv6. El campo slug en incidents es para URLs amigables (/incidents/ransomware-hospital-bs-as). Podés generarlo desde el backend a partir del title.

CAPA 2 — Backend API

Construir la API REST en CodeIgniter 4: autenticación JWT, CRUD de incidentes, interacciones sociales y panel de admin. Cada archivo tiene una única responsabilidad.

API-01 · Setup inicial de CodeIgniter 4 como API REST pura	Backend API	CRÍTICO	2-3 hs
---	--------------------	----------------	---------------

OBJETIVO

Configurar CI4 para funcionar exclusivamente como API: sin vistas, con CORS habilitado para el frontend de React, y con la estructura de carpetas del proyecto definida.

CONTEXTO

CI4 viene configurado para aplicaciones web con vistas. Hay que "desvestirlo" para que sea una API pura. Esto incluye configurar las rutas base, el manejo de errores en formato JSON, y preparar la estructura de carpetas que vas a usar en todo el proyecto.

ARCHIVOS A CREAR / MODIFICAR

- › app/Config/App.php — baseUrl y configuración base
- › app/Config/Cors.php — orígenes permitidos (localhost:5173 de Vite)
- › app/Config/Routes.php — rutas base y grupo /api
- › app/Config/Database.php — conexión a MySQL
- › app/Filters/CorsFilter.php — filter que agrega headers CORS en cada response
- › app/Exceptions/Handler.php — errores siempre en JSON
- › app/Controllers/BaseApiController.php — clase base con helpers de response
- › .env — variables de entorno (DB, JWT secret, app env)

PASOS DETALLADOS

- › En .env configurar: CI_ENVIRONMENT=development, database.default.hostname, username, password, database. Nunca commitear este archivo.
- › Crear BaseApiController que extienda ResourceController. Este controller base va a tener dos métodos privados que van a usar todos los controllers del proyecto: success(\$data, \$statusCode = 200) que devuelve {"success": true, "data": ...} y error(\$message, \$statusCode = 400) que devuelve {"success": false, "message": ...}. Todos los controllers del proyecto extienden de este.
- › Configurar CorsFilter: el filter debe agregar los headers Access-Control-Allow-Origin (con el origen de React, no *), Access-Control-Allow-Headers (Content-Type, Authorization) y Access-Control-Allow-Methods (GET, POST, PUT, DELETE, OPTIONS). También debe manejar el preflight: si el método es OPTIONS, devolver 200 inmediatamente sin pasar al controller.
- › Registrar el CorsFilter en app/Config/Filters.php como "before" para todas las rutas.
- › En Routes.php definir el grupo base: \$routes->group("api", function(\$routes) { ... }). Todas las rutas del proyecto van dentro de este grupo.
- › Configurar el manejador de excepciones para que cualquier error no capturado devuelva JSON con el formato de error estándar, nunca HTML.
- › Crear la ruta GET /api/health que devuelva {"success": true, "data": {"status": "ok", "version": "1.0"}} para verificar que la API responde.
- › Instalar la librería de JWT. La más usada para CI4 es firebase/php-jwt vía Composer: composer require firebase/php-jwt.

CRITERIO DE ACEPTACIÓN — "esto funciona cuando..."

- › GET /api/health devuelve 200 con JSON desde Postman o Thunder Client.
- › Hacer un request desde React (con fetch) a /api/health y que no aparezca error de CORS en la consola del browser.
- › Cualquier ruta que no existe devuelve un JSON de error, no una página HTML de CI4.

NOTA

El CORS es el primer obstáculo real del proyecto. Configuralo correctamente antes de avanzar. Si después ves errores de CORS, casi siempre es el preflight OPTIONS que no está manejado o el header Authorization que no está en la lista permitida.

API-02 · Estructura base: Entity, DTO, Converter (patrón de responsabilidad única)

Backend API

CRÍTICO

3-4
hs

OBJETIVO

Crear las clases base del patrón de datos del proyecto: Entity (mapea fila de DB), RequestDTO (valida lo que entra), ResponseDTO (define lo que sale), y Converter (transforma entre ellos). Implementarlos para la entidad User como ejemplo del patrón.

CONTEXTO

Este es el patrón más importante del proyecto. Una vez que lo tenés claro con User, el resto de entidades (Incident, Comment) siguen exactamente la misma estructura. La separación existe por una razón: la Entity puede tener campos que nunca deben salir al exterior (password_hash, is_banned). El ResponseDTO garantiza que solo se expone lo que se decide exponer.

ARCHIVOS A CREAR / MODIFICAR

- › app/Entities/UserEntity.php
- › app/DTOs/User/UserRequestDTO.php
- › app/DTOs/User/UserResponseDTO.php
- › app/DTOs/User/UserPublicResponseDTO.php – versión pública (sin email, menos datos)
- › app/Converters/UserConverter.php
- › app/DTOs/Incident/IncidentRequestDTO.php
- › app/DTOs/Incident/IncidentResponseDTO.php
- › app/Entities/IncidentEntity.php
- › app/Converters/IncidentConverter.php

PASOS DETALLADOS

- › UserEntity: clase PHP con propiedades tipadas que mapean exactamente las columnas de la tabla users. Incluye: int \$id, string \$username, string \$email, string \$passwordHash, string \$role, ?string \$avatarUrl, ?string \$bio, bool \$isBanned, string \$createdAt. Esta clase NO tiene lógica, solo datos.
- › UserResponseDTO: clase con solo los campos que el frontend debe recibir. NUNCA incluir passwordHash. Incluir: int \$id, string \$username, string \$email, string \$role, ?string \$avatarUrl, ?string \$bio, string \$createdAt. Método toArray() que devuelve los datos como array asociativo para el JSON.
- › UserPublicResponseDTO: versión aún más reducida para cuando otros usuarios ven el perfil de alguien. Solo: id, username, avatarUrl. El email es privado, nunca se expone en el perfil público.
- › UserRequestDTO: clase con los campos que llegan del frontend al registrarse o actualizar perfil. Incluir: string \$username, string \$email, string \$password. Incluir un método validate() que devuelve un array de errores si algo no cumple las reglas (username mínimo 3 chars, email válido, password mínimo 8 chars). Si el array está vacío, los datos son válidos.
- › UserConverter: clase con métodos estáticos. fromEntity(UserEntity \$entity): UserResponseDTO convierte Entity a ResponseDTO. toEntity(UserRequestDTO \$dto): UserEntity convierte DTO de entrada en Entity. fromRow(array \$row): UserEntity convierte una fila raw de la DB (array asociativo) en una Entity.
- › Repetir el mismo patrón para Incident: IncidentEntity con todos los campos de la tabla, IncidentRequestDTO con los campos que manda el usuario al crear un incidente (title, description, category_id, severity, y los campos técnicos opcionales), IncidentResponseDTO con todos los datos del incidente MÁS el username del autor (join) y el conteo de likes y comentarios.
- › El IncidentResponseDTO debe tener un campo booleano opcional: bool \$isLikedByCurrentUser y bool \$isFavoritedByCurrentUser. Estos campos se populan solo cuando hay un usuario autenticado. Cuando el request es anónimo, van como false o null.

CRITERIO DE ACEPTACIÓN — "esto funciona cuando..."

- › Dado un array simulando una fila de DB, `UserConverter::fromRow()` devuelve una `UserEntity` correctamente tipada.
- › `UserConverter::fromEntity()` nunca incluye `passwordHash` en el resultado.
- › `UserRequestDTO::validate()` rechaza un email inválido y un password de menos de 8 caracteres.
- › El patrón está documentado en un comentario de clase para que sea claro al leerlo.

NOTA

No uses los Cast de CI4 ni el sistema de Entity nativo de CI4. Creá tus propias clases PHP puras. Así el patrón es explícito, portable, y no dependés de magia del framework que puede confundir.

API-03 · JWT Service completo: generar, verificar y extraer claims

Backend API

CRÍTICO

3-4
hs

OBJETIVO

Implementar el `JWTService` que maneja todo el ciclo de vida del token: generación al hacer login, verificación en cada request protegido, y extracción del `user_id` y `role` desde el token para usarlos en los Services de negocio.

CONTEXTO

JWT (JSON Web Token) es un string codificado en Base64 que tiene tres partes separadas por puntos: `header.payload.signature`. El payload contiene "claims" (datos): `user_id`, `role`, `iat` (issued at), `exp` (expiration). Está firmado con un secreto que solo el servidor conoce. Cuando llega un token, el servidor lo verifica con ese mismo secreto. Si la firma no coincide, el token fue alterado y se rechaza. Esto es lo que permite que el usuario no pueda cambiar su propio role: aunque decodifique el token (es Base64, no encryption), si modifica el payload la firma ya no coincide y el servidor lo detecta.

JWT INVOLUCRADO

Este ticket ES el servicio JWT. El secreto debe ser una cadena aleatoria larga (mínimo 32 caracteres) guardada en el `.env`, nunca hardcodeda en el código. El payload del token debe contener: `sub` (`user_id`), `role`, `username`, `iat` (timestamp actual), `exp` (timestamp actual + duración, recomendado 24hs o 7 días para desarrollo).

ARCHIVOS A CREAR / MODIFICAR

- › `app/Services/JWTService.php` — el servicio central
- › `app/Filters/JWTFilter.php` — el filter de CI4 que protege rutas
- › `app/Config/Filters.php` — registrar `JWTFilter` para las rutas protegidas

PASOS DETALLADOS

- › `JWTService::generate(UserEntity $user): string` — recibe un `UserEntity`, construye el payload con `sub=$user->id`, `role=$user->role`, `username=$user->username`, `iat=time()`, `exp=time()+86400` (24hs). Usa `JWT::encode($payload, $secret, "HS256")` de la librería `firebase/php-jwt`. Devuelve el token string.
- › `JWTService::verify(string $token): object|false` — recibe el token, llama a `JWT::decode($token, new Key($secret, "HS256"))`. Si la decodificación es exitosa devuelve el objeto con los claims. Si tira `ExpiredException`, devuelve `false`. Si tira `SignatureInvalidException`, devuelve `false`. Siempre en `try/catch`.
- › `JWTService::getUserId(string $token): int` — helper que llama a `verify()` y devuelve el claim "sub" como int. Lanzar excepción si el token no es válido.
- › `JWTService::getRole(string $token): string` — igual pero devuelve el claim "role".
- › `JWTFilter`: implementar la interfaz `FilterInterface` de CI4. En el método `before()`: extraer el header `Authorization` del request (formato "Bearer <token>"). Si no hay header, devolver response 401 con JSON `{"success": false, "message": "Token requerido"}`. Si el token existe pero es inválido o expirado, devolver 401. Si es válido, inyectar el `user_id` y el `role` en los headers del request (`$request->withAddedHeader`) para que estén disponibles en el Controller sin necesidad de decodificar de vuelta.
- › `AdminFilter`: filter adicional que verifica que el claim "role" sea "admin". Si no, devuelve 403 Forbidden. Este filter se apila después del `JWTFilter` en las rutas de `/api/admin`.
- › En `Routes.php` registrar las rutas protegidas con el filter. Ejemplo: `$routes->group("api/admin", ["filter" => "jwt,admin"], function($routes) { ... })`
- › Documentar con comentario claro en `JWTFilter` cómo acceder al `user_id` desde cualquier Controller: `$this->request->getHeaderLine("X-User-Id")`

CRITERIO DE ACEPTACIÓN — "esto funciona cuando..."

- › JWTService::generate() devuelve un string con formato xxx.xxx.xxx (tres segmentos separados por puntos).
- › JWTService::verify() devuelve false si el token fue modificado (cambiar una letra del token a mano y probar).
- › JWTService::verify() devuelve false si el token expiró (crear un token con exp=time()-1 y probar).
- › Una ruta protegida con JWTFilter devuelve 401 si no hay token en el header.
- › Una ruta con AdminFilter devuelve 403 si el token tiene role="user".
- › El user_id está disponible en el Controller via getHeaderLine sin necesidad de decodificar el token de vuelta.

NOTA

Nunca guardes el JWT en el payload información sensible como el password o datos bancarios. Solo lo mínimo: id, role, username. El token no es encryption, es firma. Cualquiera puede ver el payload si decodifica el Base64, pero no puede modificarlo sin invalidar la firma.

API-04 · AuthController y AuthService: register, login, me	Backend API	CRÍTICO	3-4 hs
--	-------------	---------	--------

OBJETIVO

Implementar los tres endpoints de autenticación: POST /api/auth/register, POST /api/auth/login, GET /api/auth/me. Son los primeros endpoints reales del sistema y el punto de entrada de todo el flujo JWT.

CONTEXTO

El register crea un usuario nuevo, hashea su password con password_hash() y devuelve el token directo (sin que tenga que hacer login aparte). El login verifica credenciales y devuelve el token. El /me es el endpoint clave para el frontend: React lo llama al iniciar con el token guardado y recibe los datos del usuario actual. Si /me devuelve 401, el usuario no está logueado.

JWT INVOLUCRADO

El endpoint /me es el más importante del sistema para la personalización. Recibe el JWT, lo decodifica con JWTFilter, y usa el user_id extraído del token para buscar al usuario en la DB. El Controller NUNCA recibe el user_id del body del request. Siempre lo saca del token. Así es imposible que un usuario solicite datos de otro usuario.

ARCHIVOS A CREAR / MODIFICAR

- › app/Controllers/AuthController.php
- › app/Services/AuthService.php
- › app/Models/UserModel.php

PASOS DETALLADOS

- › UserModel: extiende Model de CI4. Definir \$table = "users", \$allowedFields con los campos que se pueden insertar/actualizar (nunca el id, ni created_at). Métodos: findByEmail(string \$email): ?UserEntity y findById(int \$id): ?UserEntity, ambos usan UserConverter::fromRow() para devolver una Entity tipada en vez del array crudo.
- › AuthService::register(UserRequestDTO \$dto): array — valida el DTO, verifica que el email no esté en uso (si está, lanzar excepción), crea la Entity con passwordHash=password_hash(\$dto->password, PASSWORD_BCRYPT), inserta en DB, genera el JWT con JWTService::generate(), devuelve ["user" => \$responseDTO->toArray(), "token" => \$token].
- › AuthService::login(string \$email, string \$password): array — busca el usuario por email, si no existe lanzar excepción genérica ("Credenciales inválidas" — nunca decir qué campo falló por seguridad), verifica password con password_verify(), genera el JWT, devuelve igual que register.
- › AuthService::me(int \$userId): UserResponseDTO — busca el usuario por ID (que viene del token), devuelve el ResponseDTO. Si el usuario está baneado (is_banned=1), lanzar excepción 403.
- › AuthController::register(): extrae el JSON del body con \$this->request->getJSON(), crea el RequestDTO, llama a AuthService::register(), devuelve con \$this->success(...).
- › AuthController::login(): igual pero extrae email y password, llama a AuthService::login().
- › AuthController::me(): está protegido con JWTFilter. Extrae el user_id del header X-User-Id, llama a AuthService::me(\$userId), devuelve el ResponseDTO.
- › Registrar las rutas: POST /api/auth/register, POST /api/auth/login sin filter. GET /api/auth/me con filter "jwt".

CRITERIO DE ACEPTACIÓN — "esto funciona cuando..."

- › POST /api/auth/register con datos válidos devuelve 200 con el token y los datos del usuario (sin password).
- › POST /api/auth/register con email duplicado devuelve 400 con mensaje de error.
- › POST /api/auth/login con credenciales correctas devuelve el token.
- › POST /api/auth/login con password incorrecto devuelve 401 con mensaje genérico.
- › GET /api/auth/me con token válido devuelve los datos del usuario actual.
- › GET /api/auth/me sin token devuelve 401.
- › GET /api/auth/me con el token de Usuario A NUNCA puede devolver datos del Usuario B.

API-05 · IncidentController y IncidentService: CRUD completo con paginación y filtros

Backend API

ALTO

5-6
hs

OBJETIVO

Implementar el CRUD completo de incidentes: listar con paginación y filtros, ver detalle, crear, editar, y eliminar. Es el núcleo del sistema. Los endpoints de lista y detalle son públicos; crear, editar y eliminar requieren JWT.

CONTEXTO

El listado de incidentes es el endpoint más complejo del backend porque combina paginación, múltiples filtros (severidad, categoría, búsqueda por texto, CVE), y datos adicionales que requieren JOINS (username del autor, conteo de likes, conteo de comentarios). Si hay un usuario autenticado, también debe indicar si ese usuario específico ya likeó o guardó cada incidente.

JWT INVOLUCRADO

Al crear un incidente, el user_id del autor siempre se extrae del JWT, nunca del body del request. Aunque el frontend envíe un user_id en el body, el Service lo ignora y usa el del token. Al editar o eliminar, el Service verifica que el user_id del token coincida con el user_id del incidente, o que el role sea "admin". Si no, devuelve 403 Forbidden.

ARCHIVOS A CREAR / MODIFICAR

- › app/Controllers/IncidentController.php
- › app/Services/IncidentService.php
- › app/Models/IncidentModel.php

PASOS DETALLADOS

- › IncidentModel: tabla "incidents". Método findAll(array \$filters, int \$page, int \$perPage): array — construye la query con joins (LEFT JOIN users ON..., LEFT JOIN categories ON...), aplica filtros opcionales (WHERE severity=?, WHERE category_id=?, WHERE title LIKE %?%, WHERE cve_id=?), aplica paginación con LIMIT y OFFSET calculados desde \$page y \$perPage. Devuelve array con "data" (array de filas) y "total" (conteo sin paginación para que el frontend sepa cuántas páginas hay).
- › Si hay user_id disponible, el IncidentModel agrega dos subconsultas al SELECT: (SELECT COUNT(*) FROM likes WHERE incident_id=i.id AND user_id=?) AS is_liked y (SELECT COUNT(*) FROM favorites WHERE incident_id=i.id AND user_id=?) AS is_favorited. Estos son los campos que personalizan el feed para cada usuario.
- › IncidentService::getAll(array \$queryParams, ?int \$currentUserId): array — extrae y sanitiza los filtros del queryParams (page, per_page, severity, category_id, search, cve), llama al Model, convierte cada fila en IncidentResponseDTO con IncidentConverter::fromRow(), agrega is_liked e is_favorited si hay currentUserId, devuelve los DTOs paginados.
- › IncidentService::getById(int \$id, ?int \$currentUserId): IncidentResponseDTO — busca el incidente con JOIN a users y categories, incrementa views_count, devuelve el DTO.
- › IncidentService::create(IncidentRequestDTO \$dto, int \$userId): IncidentResponseDTO — genera el slug a partir del title (strtolower, reemplazar espacios por guiones, limpiar caracteres especiales), asigna el user_id del token, inserta, devuelve el DTO del incidente creado.
- › IncidentService::update(int \$id, IncidentRequestDTO \$dto, int \$userId, string \$role): IncidentResponseDTO — verifica autorización o admin, actualiza solo los campos permitidos, devuelve el DTO actualizado.
- › IncidentService::delete(int \$id, int \$userId, string \$role): void — verifica autorización o admin, elimina (o hace soft delete cambiando status a "archived").

- › IncidentController: métodos index() para el listado, show(\$id) para el detalle. El user_id es opcional en ambos: si hay token válido se personaliza el resultado, si no hay token se devuelve igual pero sin is_liked e is_favorited. Para el token opcional, no usar JWTFilter (que bloquea si no hay token), sino leer el header manualmente en el Controller y parsear si existe.
- › Registrar rutas: GET /api/incidents (público), GET /api/incidents/{id} (público), POST /api/incidents (jwt), PUT /api/incidents/{id} (jwt), DELETE /api/incidents/{id} (jwt).

CRITERIO DE ACEPTACIÓN — "esto funciona cuando..."

- › GET /api/incidents devuelve la primera página con los campos de paginación: data, total, page, per_page, total_pages.
- › GET /api/incidents?severity=critical filtra correctamente.
- › GET /api/incidents?search=ransomware busca por texto en title y description.
- › GET /api/incidents con token válido incluye is_liked e is_favorited para cada incidente.
- › POST /api/incidents sin token devuelve 401.
- › PUT /api/incidents/{id} con el token de otro usuario (que no sea admin) devuelve 403.
- › El campo user_id del incidente siempre corresponde al del token, no al del body.

API-06 · Interacciones: likes, favoritos y comentarios	Backend API	ALTO	3-4 hs
---	-------------	------	--------

OBJETIVO

Implementar los endpoints de interacción social: togglear like en un incidente, togglear favorito, crear comentario, listar comentarios de un incidente y eliminar comentario propio.

CONTEXTO

El "toggle" significa que el mismo endpoint sirve para likear y para quitarle el like. Si el registro (user_id, incident_id) existe en la tabla likes, lo borra. Si no existe, lo crea. Esto simplifica el frontend que solo necesita llamar a un endpoint y la API se encarga de decidir qué hacer.

JWT INVOLUCRADO

Todos estos endpoints requieren JWT. El user_id siempre viene del token. Un usuario solo puede borrar sus propios comentarios (o un admin puede borrar cualquiera). La query de like/favorito siempre usa el user_id del token como condición.

ARCHIVOS A CREAR / MODIFICAR

- › app/Controllers/LikeController.php
- › app/Controllers/FavoriteController.php
- › app/Controllers/CommentController.php
- › app/Services/LikeService.php
- › app/Services/FavoriteService.php
- › app/Services/CommentService.php
- › app/Models/LikeModel.php
- › app/Models/FavoriteModel.php
- › app/Models/CommentModel.php

PASOS DETALLADOS

- › LikeService::toggle(int \$incidentId, int \$userId): array — busca en la tabla likes el registro donde user_id=\$userId AND incident_id=\$incidentId. Si existe, lo elimina y devuelve ["liked" => false, "count" => nuevo_total]. Si no existe, lo inserta y devuelve ["liked" => true, "count" => nuevo_total]. El conteo total de likes se recalcula con COUNT en la tabla.
- › FavoriteService::toggle(int \$incidentId, int \$userId): array — igual lógica que likes pero sobre la tabla favorites.
- › CommentService::create(int \$incidentId, string \$content, int \$userId): CommentResponseDTO — valida que content no esté vacío y tenga menos de 2000 chars, inserta el comentario, devuelve el DTO con los datos del comentario más el username del autor.

- › `CommentService::getByIncident(int $incidentId)`: array — devuelve todos los comentarios no eliminados de un incidente, ordenados por `created_at` ASC, cada uno con el `username` y `avatarUrl` del autor.
- › `CommentService::delete(int $commentId, int $userId, string $role)`: void — verifica que el comentario existe, que el `user_id` sea el del autor o que el `role` sea "admin". Hace soft delete: `is_deleted=1`.
- › Rutas: `POST /api/incidents/{id}/like (jwt)`, `POST /api/incidents/{id}/favorite (jwt)`, `POST /api/incidents/{id}/comments (jwt)`, `GET /api/incidents/{id}/comments (público)`, `DELETE /api/comments/{id} (jwt)`.

CRITERIO DE ACEPTACIÓN — "esto funciona cuando..."

- › Llamar dos veces a `POST /api/incidents/1/like`: la primera vez `liked=true`, la segunda `liked=false`.
- › `GET /api/incidents/1/comments` devuelve los comentarios con `username` del autor.
- › `DELETE /api/comments/{id}` con el token del autor devuelve 200.
- › `DELETE /api/comments/{id}` con el token de otro usuario devuelve 403.

API-07 · Dashboard del usuario: mis incidentes, mis favoritos, mis stats

Backend API

ALTO

2-3
hs

OBJETIVO

Implementar los endpoints del dashboard personal: lista de incidentes propios, lista de favoritos guardados, y estadísticas personales (total de posts, total de likes recibidos, etc.).

CONTEXTO

Estos endpoints son los que convierten al sistema en algo personal para cada usuario. Toda la información que devuelven es filtrada por el `user_id` del token. Nunca hay un parámetro de "dame el dashboard del user X", porque el dashboard es siempre del usuario autenticado.

JWT INVOLUCRADO

Estos endpoints son los que mejor demuestran el valor del JWT para datos únicos. `GET /api/dashboard/incidents` devuelve SOLO los incidentes del `user_id` del token. `GET /api/dashboard/favorites` devuelve SOLO los favoritos del `user_id` del token. Las queries siempre tienen `WHERE user_id = ?` con el ID extraído del token.

ARCHIVOS A CREAR / MODIFICAR

- › `app/Controllers/DashboardController.php`
- › `app/Services/DashboardService.php`

PASOS DETALLADOS

- › `DashboardService::getMyIncidents(int $userId, int $page)`: array — igual que `IncidentModel::findAll()` pero con `WHERE user_id=$userId` fijo. El usuario ve sus propios incidentes con paginación.
- › `DashboardService::getMyFavorites(int $userId, int $page)`: array — JOIN entre favoritos y incidents `WHERE favorites.user_id=$userId`. Devuelve los incidentes favoritos del usuario.
- › `DashboardService::getMyStats(int $userId)`: array — devuelve un objeto con: `total_incidents` (COUNT de mis posts), `total_likes_received` (COUNT de likes en todos mis incidentes), `total_comments_received` (COUNT de comentarios en todos mis incidentes), `total_favorites_given` (COUNT de mis favoritos). Son cuatro queries simples con GROUP BY o subconsultas.
- › `DashboardController`: todos los métodos protegidos con `JWTFilter`. Extraen el `user_id` del header `X-User-Id`.
- › Rutas con filter "jwt": `GET /api/dashboard/incidents`, `GET /api/dashboard/favorites`, `GET /api/dashboard/stats`.

CRITERIO DE ACEPTACIÓN — "esto funciona cuando..."

- › `GET /api/dashboard/incidents` devuelve solo los incidentes del usuario del token, no todos.
- › `GET /api/dashboard/favorites` devuelve los incidentes que el usuario marcó como favorito.
- › `GET /api/dashboard/stats` devuelve un objeto con los cuatro contadores correctamente calculados.
- › Con el token del Usuario A es imposible ver el dashboard del Usuario B.

OBJETIVO

Implementar los endpoints del panel de administración: listar todos los usuarios, banear/desbanear un usuario, destacar un incidente, forzar eliminación de cualquier incidente o comentario.

CONTEXTO

Todos estos endpoints están protegidos con JWTFilter + AdminFilter. El AdminFilter verifica que el claim "role" del token sea "admin". Si cualquier usuario normal intenta acceder a /api/admin/*, recibe 403 automáticamente sin que el Controller siquiera se ejecute.

JWT INVOLUCRADO

La doble capa de filtros (JWT + Admin) es el mecanismo de seguridad. JWT garantiza que hay un usuario autenticado. Admin garantiza que ese usuario tiene el role correcto. Estos claims están en el token y fueron definidos al momento del login, firmados por el servidor. El usuario no puede cambiar su propio role.

ARCHIVOS A CREAR / MODIFICAR

- › app/Controllers/Admin/AdminUserController.php
- › app/Controllers/Admin/AdminIncidentController.php
- › app/Services/Admin/AdminUserService.php
- › app/Services/Admin/AdminIncidentService.php

PASOS DETALLADOS

- › AdminUserService::getAllUsers(int \$page): array — lista todos los usuarios con paginación, incluyendo is_banned y role. Solo visible para admins.
- › AdminUserService::toggleBan(int \$targetUserId): array — cambia is_banned a 1 si era 0, a 0 si era 1. Devuelve el nuevo estado. Verificar que no se pueda banear a otro admin.
- › AdminIncidentService::toggleFeatured(int \$incidentId): array — cambia is_featured entre 0 y 1. Los incidentes destacados aparecen primero en el feed.
- › AdminIncidentService::forceDelete(int \$incidentId): void — elimina el incidente sin importar quién sea el autor.
- › Registrar rutas bajo el grupo api/admin con filter "jwt,admin": GET /api/admin/users, POST /api/admin/users/{id}/ban, POST /api/admin/incidents/{id}/feature, DELETE /api/admin/incidents/{id}.

CRITERIO DE ACEPTACIÓN — "esto funciona cuando..."

- › GET /api/admin/users con token de usuario normal devuelve 403.
 - › POST /api/admin/users/{id}/ban con token de admin cambia el estado del usuario.
 - › Un usuario baneado (is_banned=1) que intenta hacer login recibe 403 con mensaje claro.
 - › DELETE /api/admin/incidents/{id} con token de admin elimina el incidente aunque no sea suyo.
-

CAPA 3 — Frontend React

Construir el frontend en React: páginas, componentes, contexto de auth y consumo de la API. La estética es parte del entregable.

FE-01 · Setup React: estructura, Axios, contexto de Auth y rutas

Frontend React

CRÍTICO

3-4
hs

OBJETIVO

Configurar el proyecto React con Vite, instalar Axios, crear el contexto global de autenticación y configurar React Router con las rutas públicas, protegidas y de admin.

CONTEXTO

El AuthContext es el corazón del frontend. Es el lugar donde vive el JWT y los datos del usuario actual. Todos los componentes que necesiten saber "¿hay alguien logueado?" o "¿cuál es el usuario actual?" lo leen de este contexto. Axios tiene un interceptor que agrega automáticamente el token a cada request.

JWT INVOLUCRADO

El JWT se guarda en localStorage (para persistir entre recargas) o en memoria de React (más seguro pero se pierde al recargar). Para este proyecto, localStorage es aceptable. Al iniciar la app, se lee el token del localStorage y se llama a /api/auth/me para verificar que sigue siendo válido. Si /me devuelve 401, se limpia el token y el usuario queda como no autenticado.

ARCHIVOS A CREAR / MODIFICAR

- › src/main.jsx — entry point
- › src/App.jsx — router principal
- › src/context/AuthContext.jsx — contexto global de auth
- › src/services/api.js — instancia de Axios con baseURL e interceptores
- › src/components/ProtectedRoute.jsx — wrapper de rutas que requieren login
- › src/components/AdminRoute.jsx — wrapper de rutas que requieren role admin
- › .env — VITE_API_URL=http://localhost:8080/api

PASOS DETALLADOS

- › Crear el proyecto con Vite: `npm create vite@latest cyber-incident-hub -- --template react`. Instalar dependencias: `npm install axios react-router-dom`.
- › api.js: crear instancia de axios con baseURL desde `import.meta.env.VITE_API_URL`. Agregar request interceptor que lee el token de localStorage y agrega el header `Authorization: "Bearer " + token` a cada request si el token existe. Agregar response interceptor que, si recibe un 401, limpia el token del localStorage y redirige al login.
- › AuthContext: crear con `createContext`. El AuthProvider maneja el estado: user (objeto con id, username, role, etc.), token (string), isLoading (boolean para el check inicial). El método `login(email, password)` llama a `POST /api/auth/login`, guarda el token en localStorage y en el estado, carga los datos de `/api/auth/me` en user. El método `logout()` limpia el localStorage y el estado. Al montar el Provider (`useEffect` vacío), leer el token del localStorage y llamar a `/api/auth/me`: si responde bien, setear el user; si da 401, limpiar.
- › ProtectedRoute: componente que recibe children. Si isLoading, muestra un spinner. Si no hay user, redirige a /login con `Navigate`. Si hay user, renderiza los children.
- › AdminRoute: igual que ProtectedRoute pero además verifica que `user.role === "admin"`. Si no, redirige a / (home).
- › App.jsx: configurar rutas con `BrowserRouter` y `Routes`. Rutas públicas: / (Feed), /incidents/:id (Detalle), /login, /register. Rutas protegidas (dentro de ProtectedRoute): /dashboard. Rutas admin (dentro de AdminRoute): /admin.

CRITERIO DE ACEPTACIÓN — "esto funciona cuando..."

- › Al hacer login exitoso, el token aparece en localStorage.
- › Al recargar la página, el usuario sigue logueado (el token persiste y /me funciona).
- › Intentar entrar a /dashboard sin login redirige automáticamente a /login.
- › Intentar entrar a /admin con un usuario de role "user" redirige a /.
- › Hacer logout limpia el localStorage y el estado del contexto.

- › Todos los requests que salen con Axios tienen el header Authorization si el usuario está logueado.

FE-02 · Páginas de Auth: Login y Register con diseño SOC Dashboard

Frontend React

ALTO

3-4
hs

OBJETIVO

Crear las páginas de Login y Register con la estética "terminal/SOC Dashboard": dark mode, fuente monoespaciada, severity badges y la sensación de estar en una plataforma de seguridad real.

CONTEXTO

El diseño es parte del proyecto. No es solo funcionalidad. La identidad visual del Cyber Incident Hub debe transmitir profesionalismo técnico. Las páginas de auth son lo primero que ve un usuario nuevo.

ARCHIVOS A CREAR / MODIFICAR

- › `src/pages/LoginPage.jsx`
- › `src/pages/RegisterPage.jsx`
- › `src/components/ui/FormInput.jsx` — input reutilizable
- › `src/components/ui/AlertBanner.jsx` — errores y mensajes
- › `src/styles/theme.css` — variables CSS del design system

PASOS DETALLADOS

- › Definir en `theme.css` las variables del design system: `--bg-primary (#0D1117)`, `--bg-secondary (#161B22)`, `--bg-surface (#21262D)`, `--accent (#58A6FF)`, `--danger (#FF4444)`, `--warning (#FF8800)`, `--success (#33CC66)`, `--text-primary (#E6EDF3)`, `--text-muted (#8B949E)`, `--border (#30363D)`, `--font-mono ('JetBrains Mono', 'Fira Code', monospace)`.
- › `LoginPage`: formulario centrado en la pantalla con fondo `--bg-primary`. Logo o nombre del sistema arriba con fuente monoespaciada y el color `--accent`. Dos campos: email y password. Botón de submit con el texto "> AUTHENTICATE". Manejo de estado local con `useState` para email, password, error, `isLoading`. Al submit: llama a `AuthContext.login()`, si hay error muestra `AlertBanner` con el mensaje del 401.
- › `RegisterPage`: igual estética pero con campo adicional de username. Al registrarse exitosamente, redirigir directo al feed (no al login).
- › `FormInput`: componente reutilizable con label, input, y mensaje de error inline. El estilo debe verse como un terminal: borde en `--border`, background `--bg-surface`, texto `--text-primary`, font-family `--font-mono`, sin bordes redondeados agresivos.
- › `AlertBanner`: muestra mensajes de error (rojo con `border-left` en `--danger`) o éxito (verde). Tiene un botón X para cerrarlo.
- › Agregar un link "¿No tenés cuenta? Registrate" en Login y viceversa en Register.

CRITERIO DE ACEPTACIÓN — "esto funciona cuando..."

- › Login exitoso redirige al feed y el usuario aparece en el navbar.
- › Login con credenciales incorrectas muestra el mensaje de error sin romper la app.
- › Register con email duplicado muestra el error específico.
- › La estética es consistente: todo dark, fuente mono, sin colores brillantes excepto el accent azul.

FE-03 · Feed principal: listado de incidentes con filtros y paginación

Frontend React

ALTO

4-5
hs

OBJETIVO

Crear el feed principal de incidentes con los severity badges de colores, filtros por severidad y categoría, buscador de texto/CVE, y paginación. Es la página más importante del sistema visualmente.

ARCHIVOS A CREAR / MODIFICAR

- › src/pages/FeedPage.jsx
- › src/components/incidents/IncidentCard.jsx
- › src/components/incidents/SeverityBadge.jsx
- › src/components/incidents/FilterBar.jsx
- › src/components/ui/Pagination.jsx
- › src/hooks/useIncidents.js – custom hook para el fetching

PASOS DETALLADOS

- › useIncidents hook: maneja el estado de filters (objeto con severity, category_id, search, page), hace el fetch a /api/incidents con los filtros como query params usando Axios, maneja loading y error states, devuelve los datos y los setters.
- › SeverityBadge: componente que recibe severity string y devuelve un badge coloreado. critical → rojo (#FF4444) con texto blanco. high → naranja (#FF8800). medium → amarillo (#FFCC00) con texto negro. low → verde (#33CC66). El badge debe ser pequeño, pill shape, con fuente monoespaciada en mayúsculas.
- › IncidentCard: tarjeta que muestra título, SeverityBadge, categoría, username del autor, fecha formateada, conteo de likes y comentarios, y los primeros 150 chars de la descripción. Si el usuario está autenticado, mostrar un botón de like que cambia de estado visualmente al hacer click (optimistic update: cambia el ícono inmediatamente, luego manda el request).
- › FilterBar: row con tres controles. Dropdown de severidad (All, Critical, High, Medium, Low). Dropdown de categoría (cargado desde GET /api/categories). Input de búsqueda con debounce de 400ms (no buscar en cada keystroke). Al cambiar cualquier filtro, resetear a page=1.
- › FeedPage: composición de FilterBar + lista de IncidentCards + Pagination. Mostrar un skeleton loader (placeholders grises animados) mientras carga. Si no hay resultados, mostrar un mensaje estilo terminal: "NO INCIDENTS FOUND — MODIFY SEARCH PARAMETERS".
- › Pagination: botones Anterior/Siguiente con el número de página actual y total. Deshabilitar el botón cuando no corresponde (página 1 no tiene anterior).

CRITERIO DE ACEPTACIÓN — "esto funciona cuando..."

- › El feed carga y muestra incidentes al entrar a /.
- › Cambiar el filtro de severidad actualiza la lista sin recargar la página.
- › Buscar por texto hace el fetch después del debounce, no en cada letra.
- › El like funciona con actualización visual inmediata.
- › La paginación navega correctamente entre páginas.
- › Si el usuario no está logueado, el botón de like muestra "Login to like" o redirige al login.

FE-04 · Detalle de incidente, dashboard personal y admin panel

Frontend React

ALTO

5-6
hs

OBJETIVO

Crear la página de detalle de un incidente (con todos los datos técnicos y la sección de comentarios), el dashboard personal del usuario, y el panel de admin con la lista de usuarios y moderación.

ARCHIVOS A CREAR / MODIFICAR

- › src/pages/IncidentDetailPage.jsx
- › src/pages/DashboardPage.jsx
- › src/pages/AdminPage.jsx
- › src/components/incidents/CommentSection.jsx
- › src/components/incidents/TechnicalDataPanel.jsx
- › src/components/dashboard/StatsCard.jsx
- › src/components/admin/UserTable.jsx

PASOS DETALLADOS

- › IncidentDetailPage: usa el id de los params de React Router. Fetch a `/api/incidents/:id`. Muestra: título grande, SeverityBadge grande, categoría, autor con link al perfil, fecha, datos técnicos en TechnicalDataPanel (IP, dominio, CVE con link a `nvd.nist.gov` si hay, malware, país), descripción completa, tags, botones de like y favorito, y debajo la sección de comentarios.
- › TechnicalDataPanel: grilla de campos técnicos estilo "terminal readout". Cada campo que existe se muestra en una fila label: valor con fuente mono. Si un campo es null, no se muestra (no mostrar "CVE: —").
- › CommentSection: muestra la lista de comentarios ordenados por fecha. Si el usuario está logueado, muestra un textarea para agregar comentario con botón submit. Cada comentario muestra avatar (o inicial del username si no hay avatar), username, fecha relativa ("hace 2 horas"), y el contenido. Si el comentario es del usuario actual, mostrar botón de eliminar.
- › DashboardPage: tres secciones con tabs o secciones verticales. Stats arriba: cuatro StatsCard con `total_incidents`, `total_likes_received`, `total_comments_received`, `total_favorites_given`. Abajo: lista de mis incidentes (igual que el feed pero solo los propios) con opción de editar o eliminar cada uno. Y la lista de mis favoritos.
- › AdminPage: solo accesible con AdminRoute. Tabla de todos los usuarios con columnas: username, email, role, `is_banned`, fecha de registro, y botón de acción (Banear/Desbanear). El botón hace POST a `/api/admin/users/{id}/ban` y actualiza el estado local.
- › Agregar navbar con: logo del sistema, links de navegación, y si el usuario está logueado: su username, link al dashboard, y el botón de logout. Si es admin, mostrar link a `/admin`.

CRITERIO DE ACEPTACIÓN — "esto funciona cuando..."

- › La página de detalle muestra correctamente todos los campos técnicos que existen, sin mostrar los que son null.
 - › Agregar un comentario lo muestra en la lista inmediatamente (sin recargar).
 - › El dashboard muestra las stats correctas del usuario logueado.
 - › El admin puede banear un usuario y el estado se actualiza en la tabla.
 - › Un usuario baneado que intenta hacer login recibe un mensaje claro.
-

CAPA 4 — Integración

Conectar las capas, testear los flujos end-to-end y pulir los detalles que hacen la diferencia.

INT-01 · Testing de integración end-to-end y manejo de errores global

Integración

ALTO

3-4
hs

OBJETIVO

Recorrer todos los flujos del sistema de punta a punta, verificar el manejo de errores en cada capa, y asegurar que los mensajes de error lleguen correctamente al usuario en el frontend.

PASOS DETALLADOS

- › Flujo completo de auth: register → login → /me → acceder a dashboard → logout → verificar que el dashboard ya no es accesible.
- › Flujo de incidente: crear incidente logueado → verlo en el feed → likearlo → guardarlo como favorito → verlo en favoritos del dashboard → editarlo → eliminarlo.
- › Flujo de comentarios: agregar comentario → verlo en el detalle → eliminarlo.
- › Flujo de admin: loguearse como admin → ver lista de usuarios → banear un usuario → intentar loguearse con el usuario baneado → desbanear → destacar un incidente → verificar que aparece primero en el feed.
- › Manejo de errores: desconectar la DB y verificar que la API devuelve JSON de error (no HTML). Enviar un token expirado y verificar el redirect al login en el frontend. Intentar acceder a un incidente que no existe y verificar el 404.
- › Verificar que el token de un usuario no puede acceder a datos de otro usuario en ningún endpoint.

CRITERIO DE ACEPTACIÓN — "esto funciona cuando..."

- › Todos los flujos principales funcionan sin errores.
- › Ningún endpoint devuelve datos de otro usuario.
- › Los errores se muestran correctamente en el frontend, nunca como páginas en blanco.
- › El sistema se comporta correctamente cuando el token expira durante una sesión activa.

INT-02 · Pulido final: UX, performance y detalles que hacen la diferencia

Integración

MEDIO

3-4
hs

OBJETIVO

Agregar los detalles que convierten un proyecto funcional en una joya: loading states, empty states, confirmaciones antes de acciones destructivas, fechas relativas, y consistencia visual en toda la app.

PASOS DETALLADOS

- › Agregar skeleton loaders en todos los lugares donde hay fetch: el feed, el detalle, el dashboard. El skeleton debe tener la misma forma que el contenido real (mismo alto de tarjeta, mismas proporciones).
- › Agregar empty states descriptivos cuando una lista está vacía: "No incidents yet. Be the first to report one." con un botón de acción si aplica.
- › Agregar confirmación antes de acciones destructivas (eliminar incidente, eliminar comentario, banear usuario): un modal o confirm() simple que diga exactamente qué va a pasar.
- › Formatear todas las fechas como tiempo relativo: "hace 3 minutos", "hace 2 días", usando la librería date-fns (npm install date-fns).
- › Agregar un indicador visual cuando un incidente tiene is_featured=1 (una estrella dorada o badge de "FEATURED" en la tarjeta).
- › Revisar la consistencia del design system: que todos los inputs, botones y badges usen las mismas variables CSS. Que el espaciado sea consistente.
- › Agregar manejo del estado de la request con toasts de notificación (una pequeña notificación que aparece y desaparece) para acciones exitosas como "Incident created" o "Comment deleted". Podés usar react-hot-toast (npm install react-hot-toast).

CRITERIO DE ACEPTACIÓN — "esto funciona cuando..."

- › Ninguna parte de la app muestra una pantalla en blanco mientras carga.
 - › Todas las fechas están en formato relativo.
 - › Las acciones destructivas tienen confirmación.
 - › Las acciones exitosas muestran feedback visual.
 - › El design system es 100% consistente: mismos colores, misma fuente, mismo espaciado en toda la app.
-

Notas finales del proyecto

- › Orden de trabajo: empezar SIEMPRE por la DB. Sin el schema correcto, todo lo que viene encima tambalea.
- › JWT primero, todo lo demás después: una vez que API-03 funciona, el resto de los endpoints protegidos son fáciles de agregar.
- › Probá con Postman o Thunder Client antes de conectar el frontend. Así sabés con certeza qué devuelve la API.
- › El patrón Entity/DTO/Converter se repite igual para cada entidad. Hacelo bien con User y el resto es copy-paste con ajustes.
- › Los errores de CORS aparecen el primer día y nunca más. Una vez configurado correctamente, olvidate.
- › Guardá el JWT secret en el .env y asegurate de que el .env esté en el .gitignore.
- › Cuando algo no funciona, revisá: 1) La respuesta de la API en la Network tab. 2) El token en localStorage. 3) El header Authorization que manda Axios.